Agile & DevOps Practical Exam — Step-by-Step Manual

Based on your own journal procedures (Rashmit Gaikwad · Roll No. 10 · TE AI&DS · Course 2015119) · Experiments 1, 2, 3, 4 & 7

Contents

Exp	Title	What you actually do
1	Agile project lifecycle using Jira with DevOps pipeline integration	Jira Scrum space + sprint → GitHub repo + GitHub Actions CI → Jira–GitHub connection → SCRUM-key branches, PRs, merge
2	Version control operations using Git	Install Git on Windows, create GitHub account, init/add/commit/branch/merge/push/pull/clone
3	CI pipeline using Jenkins / GitHub Actions	Jenkins Freestyle jobs on Windows: Examples 1 – 5 (batch, Java, String/Choice parameters, Python)
4	Containerize applications using Docker	Install Docker Desktop + WSL 2, pull/run Ubuntu, ps, stop, rm, image rm
7	Deploy a serverless function using AWS Lambda	Create Python function, Deploy, test event, CloudWatch logs

Do these before exam day

Check	Why
Log in once to Jira, GitHub, AWS on the PC you'll use; know your passwords	Password resets and email codes waste exam time.
git push works (browser sign-in done once)	Needed for Exp 1 and 2.
Jenkins opens at localhost:8080; `java -version`, `javac -version`, `python --version` work in CMD	Exp 3 fails if Java/Python aren't on PATH.
C:\jenkinsprojects has hello.py, example.bat, example2.java, example3.bat, mul.java, example5.py	Copy them from this manual — no typing under pressure.
Docker Desktop shows *Engine running*; `docker pull ubuntu` already done	Docker start-up and image download take minutes.
Your GitHub repo student-management-devops still has the workflow and code	You can demo directly and only redo the branch/PR part.

In the exam: say what each step does while you do it ("this creates a feature branch named with the Jira key so Jira can link it…"). Understanding marks are 5/20 in your rubric — explaining scores as much as the output. If something errors, read it aloud and use the "Common errors" table.

Experiment 1

Agile Software Project Lifecycle using Jira with DevOps Pipeline Integration

Aim: To study and implement the Agile Software Project Lifecycle using Jira by managing Scrum tasks, tracking development progress, integrating GitHub for version control, and implementing a DevOps CI pipeline using GitHub Actions.

Workflow: Jira work item → GitHub feature branch → Commit → Push → Pull Request → GitHub Actions CI → Merge → Jira status Done

Requirements

- Atlassian (Jira) account — free plan. Newer Jira calls projects **Spaces** and issues **work items**.
- GitHub account, Git (Git Bash), Python 3 and VS Code on the PC.
- Project used in your journal: **Student Management System** (Jira) + repo **student-management-devops** (GitHub).

Procedure

Part A — Jira: create the Scrum space and backlog

1 Log in and create a Scrum space

Go to your Jira site (e.g. `yourname.atlassian.net`) → left sidebar **Spaces** → **+** (Create space) → template **Scrum** → **Use template** → Name: `Student Management System` → Jira gives the key (yours was **SCRUM**) → **Create**.

> **Tip:** Every work item gets the key as prefix: SCRUM-1, SCRUM-2 … Your numbers may differ in the exam — always use the key Jira shows you.

2 Create Scrum work items (user stories and tasks)

Open **Backlog** → **+ Create** → type **Story/Task**. Create items like in your journal:

```
Story : As a student, I want to register an account so that I can access the system
Task  : Student login
Task  : Student dashboard
Task  : Student profile management
Task  : Student search
Task  : Write unit tests
Task  : Configure CI pipeline
Task  : Add student email validation
```

Open an item to add a **description / acceptance criteria**, **assignee**, **priority** and **due date**.

3 Create and start the sprint

In Backlog click **Create sprint** → ⋯ **Edit sprint** → Name `Student management`, duration 1–2 weeks, Sprint goal: `Implement core student management features and establish automated testing and CI pipeline integration.` → drag the work items into the sprint → **Start sprint**.

4 Track progress on the board

Open **Board**. Columns: **To Do → In Progress → In Review → Done**. Drag an item (e.g. *Student login*) to **In Progress** — this is how the team tracks work daily.

Part B — GitHub: create the repository and the CI pipeline

5 Create the repository

GitHub → **New repository** → name `student-management-devops` → Public → tick **Add a README file** → **Create repository**. Then in Git Bash:

```
git clone https://github.com/<your-username>/student-management-devops.git
cd student-management-devops
code .            # opens the folder in VS Code
```

6 Add the application, tests and requirements

Create these files in the repo folder:

`app.py`

```
students = []


def add_student(student_id, name, department):
    student = {"id": student_id, "name": name, "department": department}
    students.append(student)
    return student


def get_students():
    return students


def find_student(student_id):
    for student in students:
        if student["id"] == student_id:
            return student
    return None


if __name__ == "__main__":
    add_student(1, "Rashmit", "AI & DS")
    print(get_students())
```

`test_app.py`

```
from app import add_student, get_students, find_student


def test_add_student():
    s = add_student(101, "Amit", "AI & DS")
    assert s["name"] == "Amit"
    assert s in get_students()


def test_find_student():
    add_student(102, "Neha", "Computer")
    assert find_student(102)["name"] == "Neha"
    assert find_student(999) is None
```

`requirements.txt`

```
pytest
```

Test locally first:

```
pip install -r requirements.txt
python app.py      # [{'id': 1, 'name': 'Rashmit', 'department': 'AI & DS'}]
pytest -v          # 2 passed
```

7 Add the GitHub Actions CI workflow

Create folders `.github/workflows/` and the file `student-ci.yml`:

`.github/workflows/student-ci.yml`

```
name: Student Management CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout source code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Run tests
        run: pytest -v
```

> **Watch out:** YAML uses spaces for indentation — never tabs. The folder name must be exactly `.github/workflows`.

8 Commit, push and check the pipeline

```
git add .
git commit -m "Add unit tests and CI pipeline"
git push origin main
```

On GitHub open the **Actions** tab → workflow **Student Management CI** → run **Add unit tests and CI pipeline** → job **test**.

> **Expected output:** green ✓ on every step: Set up job → Checkout source code → Set up Python → Install dependencies → Run tests → Complete job.

Part C — Integrate GitHub with Jira

9 Connect GitHub for Atlassian

In Jira open the space → **Development** tab (top) → **Connections** → under *Source code management* click **+** → choose **GitHub** (app name **GitHub for Atlassian**) → **Connect GitHub organization** → sign in to GitHub → **Install** the app on your account/organization and select `student-management-devops` → back in Jira it shows **Connected**.

> **Tip:** Older Jira UI: ⚙ Settings → Apps → Explore more apps → search “GitHub for Jira” → Get app → Connect. Same result.

Part D — Feature branch → Pull Request → Merge (per Jira work item)

10 Create a feature branch named with the Jira key (SCRUM-5)

```
git checkout main
git pull
git checkout -b SCRUM-5-student-validation
```

Move **SCRUM-5** to **In Progress** on the Jira board.

11 Implement the task — student name validation

In `app.py` replace `add_student` with:

app.py (SCRUM-5 change)

```
def add_student(student_id, name, department):
    if not name or not name.strip():
        raise ValueError("Student name cannot be empty")
    student = {"id": student_id, "name": name.strip(), "department": department}
    students.append(student)
    return student
```

In `test_app.py` add:

test_app.py (SCRUM-5 test)

```
import pytest   # put this line at the top of test_app.py


def test_empty_name_rejected():
    with pytest.raises(ValueError):
        add_student(103, "   ", "IT")
```

Run `pytest -v` → 3 passed.

12 Commit with the Jira key and push

```
git add .
git commit -m "SCRUM-5 Add student validation"
git push -u origin SCRUM-5-student-validation
```

13 Create the Pull Request

On GitHub click **Compare & pull request** (or Pull requests → New) → **base: main ← compare: SCRUM-5-student-validation** → it shows *Able to merge* → title `SCRUM-5 Add student validation` → **Create pull request**. The CI workflow runs automatically on the PR — wait for the green ✓ *All checks have passed*.

14 Merge and verify in Jira

Click **Merge pull request → Confirm merge** (optionally *Delete branch*). In Jira open **SCRUM-5 → Development** panel shows the branch, commit and pull request. Space → **Development → Pull requests** shows `SCRUM-5 Add student validation` with status **MERGED**. Move SCRUM-5 to **Done**.

15 Repeat for the second task — email validation (SCRUM-12)

```
git checkout main
git pull
git checkout -b SCRUM-12-student-email-validation
```

add to app.py

```
import re   # put this line at the top of app.py


def is_valid_email(email):
    pattern = r"^[\w.+-]+@[\w-]+\.[\w.]+$"
    return re.match(pattern, email) is not None
```

add to test_app.py

```
from app import is_valid_email   # add to the imports


def test_email_validation():
    assert is_valid_email("student@vcet.edu.in")
    assert not is_valid_email("student-at-vcet")
```

```
pytest -v                                   # 4 passed
git add .
git commit -m "SCRUM-12 Add student email validation"
git push -u origin SCRUM-12-student-email-validation
```

Create PR → checks pass → **Merge** → mark **SCRUM-12 Done** in Jira.

16 Finish the sprint

Board → **Complete sprint**. Show **Summary** / **Reports** (burndown, velocity) if the examiner asks for progress tracking.

Show the examiner:

- Jira space with backlog, sprint “Student management” and board columns.
- GitHub Actions run *Student Management CI* with all steps green.
- Jira Development → Connections showing GitHub for Atlassian **Connected**.
- A PR named with the Jira key, merged, and shown as MERGED in Jira; the work item in Done.

Common errors & fixes

Problem	Fix
Workflow not visible in Actions tab	File must be in .github/workflows/ on the main branch and the YAML indentation must be correct.
Branch / PR not appearing in Jira	Key must be in UPPERCASE in branch name/commit (SCRUM-5), the GitHub app must be installed on that repo; wait 1–2 min.
git push asks for password and fails	Use the browser sign-in window (Git Credential Manager) or a GitHub Personal Access Token.
PR says “Can't automatically merge”	Conflict with main: git pull origin main into your branch, fix the conflict, commit, push.
pytest: ModuleNotFoundError: app	Run pytest from the repo root where app.py is.

Viva questions

Q. What is the role of Jira in Agile project management? (journal Q1)

Plan, track and manage Agile projects: create user stories/tasks, organise them into sprints, monitor progress on Scrum boards and manage issues across the lifecycle.

Q. Purpose of integrating a DevOps CI pipeline with Jira? (journal Q2)
Connects project management with development and testing — code changes are linked to work items and automated builds/tests verify them, improving traceability, automation and collaboration.

Q. Why name branches/commits with the Jira key?
That is how the GitHub for Atlassian app links branches, commits and PRs to the correct Jira work item.

Q. What is a Pull Request?
A request to merge changes from a feature branch into main, allowing review, discussion and CI checks before merging.

Q. Where are GitHub Actions workflows stored?
In YAML files inside .github/workflows/ in the repository.

> **Conclusion:** The Agile Software Project Lifecycle was successfully studied and implemented using Jira with DevOps pipeline integration. Scrum work items were created and tracked in Jira, GitHub was used for version control, feature branches, commits and Pull Requests, completed changes were merged into the main branch and reflected in Jira, and a CI pipeline using GitHub Actions automated the testing of the Python application.

Experiment 2

To perform version control operations using Git

> **Aim:** To understand Version Control System — install and configure Git, create a GitHub account, and perform version control operations for code sharing and version tracking.

> **Workflow:** Working directory → git add → Staging area → git commit → Local repo → git push → GitHub

Requirements

- Windows PC with internet access.
- An email ID for the GitHub account.

Procedure

Part A — Install Git on Windows (as in your journal)

1 Download

Open **https://git-scm.com/downloads** → click **Windows** → download the 64-bit installer → double-click it → click **Yes** on the User Account Control prompt.

2 Run the installer (keep defaults)

Next on the GNU license → keep default install location → keep default components → Next → choose your **default editor** (e.g. VS Code) → initial branch name: leave default → PATH: keep the **middle (recommended)** option → SSH: use bundled OpenSSH → HTTPS: OpenSSL library → line endings: default → terminal: **MinTTY** → `git pull` behaviour: default → credential helper: **Git Credential Manager** → extra options: defaults → **Install** → tick *Launch Git Bash* → **Finish**.

3 Open Git Bash and verify

Start menu → type **git bash** → Enter:

```
git --version          # git version 2.x.x.windows.x
```

Part B — Create a GitHub account

4 Sign up

Go to **https://github.com/join** → enter email, password, username → **Verify** (solve the puzzle) → **Create account** → enter the code sent to your email → choose preferences → select the **Free** plan. Your GitHub account is ready.

Part C — Version control operations (perform these in the exam)

5 Configure your identity

```
git config --global user.name "Rashmit Gaikwad"
git config --global user.email "your-github-email@example.com"
git config --list
```

6 Create a local repository and make the first commit

```
mkdir git-demo
cd git-demo
git init                          # creates the hidden .git folder
echo "Hello Git" > file1.txt
git status                        # file1.txt is untracked (red)
git add file1.txt                 # move to staging area
git status                        # now staged (green)
git commit -m "First commit"
git log --oneline
```

7 Modify a file and see the difference

```
echo "Second line" >> file1.txt
git status                        # modified
git diff                          # shows what changed
git add .
git commit -m "Update file1"
git log --oneline
```

8 Branching and merging

```
git branch                        # list branches
git checkout -b feature           # create + switch to new branch
echo "Feature work" > feature.txt
git add .
git commit -m "Add feature file"
git checkout main                 # (use master if your default branch is master)
git merge feature                 # bring feature changes into main
git log --oneline --graph --all
git branch -d feature
```

9 Push to GitHub

On GitHub: **New repository** → name `git-demo` → Public → do **not** add a README → **Create**. Then:

```
git remote add origin https://github.com/<your-username>/git-demo.git
git branch -M main
git push -u origin main
```

A browser window asks you to sign in to GitHub the first time (Git Credential Manager). Refresh the repo page — your files and commits are online.

10 Pull and clone

Edit a file on GitHub in the browser and commit. Then:

```
git pull                          # bring the online change to your PC
cd ..
git clone https://github.com/<your-username>/git-demo.git demo-copy
```

11 Undo / revert (if asked)

```
git restore file1.txt             # discard uncommitted changes in a file
git revert HEAD                   # new commit that undoes the last commit
git stash / git stash pop         # temporarily save uncommitted work
```

Show the examiner:

- git --version output and Git Bash open.
- Your GitHub account / profile page.
- git status, git log --oneline --graph output and the merged branch.
- The repository on GitHub after git push.

Common errors & fixes

Problem	Fix
“git is not recognized”	Git not on PATH — reinstall with the recommended PATH option, or use Git Bash.
“Please tell me who you are” on commit	Run the two git config --global commands.
“src refspec main does not match any”	Nothing committed yet, or your branch is called master — use git branch -M main.
push rejected (fetch first)	Remote has newer commits: git pull, then git push.
Authentication failed	Sign in through the browser pop-up or use a Personal Access Token as password.

Viva questions

Q. What is a Version Control System? (journal Q1)
A tool that tracks and saves changes to files/source code over time, so you can see history and return to any older version.

Q. Difference between Git and GitHub? (journal Q2)
Git is the distributed version control tool on your computer; GitHub is a cloud platform that hosts Git repositories for sharing and collaboration.

Q. Centralized vs distributed VCS?
Centralized (e.g. SVN) keeps history on one server; distributed (Git) gives every developer a full copy of the history.

Q. git fetch vs git pull?
fetch only downloads changes; pull = fetch + merge.

Q. What is the staging area?
The area where you choose which changes go into the next commit (git add).

Conclusion: Git was installed and configured, a GitHub account was created, and version control operations — init, add, commit, status, log, diff, branching, merging, push, pull and clone — were performed, enabling effective collaboration, code sharing and version tracking.

Experiment 3

To set up a Continuous Integration Pipeline using Jenkins / GitHub Actions

Aim: To set up and execute a Continuous Integration pipeline using Jenkins — create Freestyle projects, run batch commands, compile and run Java programs, use parameterized builds and execute Python programs.

Workflow: New Item → Freestyle project → Build Steps (Execute Windows batch command) → Save → Build Now / Build with Parameters → Console Output

Requirements

- Jenkins on Windows (runs at **http://localhost:8080**), Java JDK (javac + java), Python, VS Code, Command Prompt.
- A working folder **C:\jenkinsprojects** containing all the programs below.
- Jenkins runs as **SYSTEM** on Windows — Java and Python must be on the **system PATH**.

Procedure

Part A — Setup (skip if Jenkins is already installed in the lab)

1 Install Java and Jenkins

Install a **JDK 17 or 21** (check: `java -version` and `javac -version` in Command Prompt). Download the **Windows LTS installer (.msi)** from **jenkins.io/download** → install with defaults (port 8080, run as LocalSystem) → open **http://localhost:8080** → paste the password from `C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword` → **Install suggested plugins** → create the admin user → **Save and Finish**.

2 Create the working folder and test programs locally

Create `C:\jenkinsprojects`, open it in VS Code and create the files used below. Always **test each program in Command Prompt first**, then run it through Jenkins.

```
cd /d C:\jenkinsprojects
python hello.py
```

hello.py

```
print("hello world")
```

Example 1 — Run a simple batch command (Freestyle project)

3 Create the job

Jenkins dashboard → **New Item** → name `Example-1` → select **Freestyle project** → **OK**.

4 Add the build step

Scroll to **Build Steps** → **Add build step** → **Execute Windows batch command** → Command:

```
cd /d C:\jenkinsprojects
python hello.py
```

→ **Save**.

5 Build and check output

Click **Build Now** → under *Builds* click **#1** → **Console Output**.

> **Expected output:** `C:\jenkinsprojects>python hello.py → hello world → Finished: SUCCESS.`

Example 1.2 — Pass parameters to a batch file

6 Write and test the batch file locally

`C:\jenkinsprojects\example.bat`

```
@echo off

set Name=%1
set Address=%2

echo Hello %Name%, your address is %Address%
```

Test in **Command Prompt** with no, one and two parameters:

```
cd /d C:\jenkinsprojects
example.bat
example.bat Bhushan
example.bat Bhushan Dombivli       # Hello Bhushan, your address is Dombivli
```

> **Watch out:** In VS Code's PowerShell terminal type `.\example.bat` — PowerShell does not run files from the current folder without `.\` (that is the red error in your journal screenshot). Command Prompt works directly.

7 Run it through Jenkins

Create Freestyle project `Example-1.2` (or click **Configure** on Example-1 to modify it) → Execute Windows batch command:

```
cd /d C:\jenkinsprojects
call example.bat Bhushan Dombivli
```

→ Save → Build Now → Console Output.

> **Expected output:** `Hello Bhushan, your address is Dombivli → Finished: SUCCESS.`

> **Tip:** Use `call` before a .bat file inside Jenkins so control returns to Jenkins after it finishes.

8 Variation (1.3) — set variables inside Jenkins

```
cd /d C:\jenkinsprojects
set name=Sachin
set Address=Dadar
call example.bat %name% %Address%
```

> **Expected output:** `Hello Sachin, your address is Dadar.`

Example 2 — Compile and run a Java program

9 Write and test the Java program

C:\jenkinsprojects\example2.java

```
public class example2 {
    public static void main(String[] args) {
        System.out.println("Hello... Welcome DevOps Class.");
    }
}
```

```
cd /d C:\jenkinsprojects
javac example2.java
java example2                       # Hello... Welcome DevOps Class.
```

10 Create the Jenkins job

New Item → `Example-2` → Freestyle project → OK → Execute Windows batch command:

```
cd /d C:\jenkinsprojects
javac example2.java
java example2
```

→ Save → Build Now → Console Output.

> **Expected output:** `Hello... Welcome DevOps Class.` → `Finished: SUCCESS`.

Example 3 — Parameterized build (String + Choice parameter)

11 Write and test the batch file

C:\jenkinsprojects\example3.bat

```
@echo off
echo Your name is %1
echo Your city is %2
```

```
example3.bat Bhushan Dombivali      # Your name is Bhushan / Your city is Dombivali
```

12 Create the parameterized job

New Item → `example 3` → Freestyle project → OK → in **General** tick **This project is parameterized** →

- **Add Parameter → String Parameter** → Name: `Fname`
- **Add Parameter → Choice Parameter** → Name: `City` → Choices (one per line):

```
Bandra
Kalyan
Dombivli
Churchgate
Thane
Dadar
```

13 Use the parameters in the build step

Build Steps → Execute Windows batch command:

```
cd /d C:\jenkinsprojects
call example3.bat %Fname% %City%
```

→ Save. The job now shows **Build with Parameters** (instead of Build Now) → enter `Fname = Rashmit`, choose `City = Bandra` → **Build** → Console Output.

Expected output: `Your name is Rashmit / Your city is Bandra` → `Finished: SUCCESS`.

Watch out: Use `%Fname%` and `%City%` — that is how Windows batch reads Jenkins parameters (on Linux it would be `$Fname`). If you hard-code names (as in your journal screenshot), the parameters you enter are ignored — examiners notice this.

Example 4 — Java program with a parameter (multiplication table)

14 Write and test the program

`C:\jenkinsprojects\mul.java`

```
class mul {
    public static void main(String[] args) {
        int i;
        int num = Integer.parseInt(args[0]);

        for (i = 1; i <= 10; i++) {
            System.out.println(num + " * " + i + " = " + (num * i));
        }
    }
}
```

```
javac mul.java
java mul 5                          # 5 * 1 = 5 ... 5 * 10 = 50
```

15 Create the parameterized job

New Item → `Example-4` → Freestyle → OK → tick **This project is parameterized** → **String Parameter** Name: `num` → Build Steps → Execute Windows batch command:

```
cd /d C:\jenkinsprojects
javac mul.java
java mul %num%
```

→ Save → **Build with Parameters** → num = `25` → Build → Console Output.

Expected output: `25 * 1 = 25 ... 25 * 10 = 250` → `Finished: SUCCESS`.

Example 5 — Python program with a parameter

16 Write and test the program

`C:\jenkinsprojects\example5.py`

```
import sys

n1 = sys.argv[1]
n2 = int(n1)

print("Original number is", n2)
print("Binary representation of", n2, "is", bin(n2))
print("Octal representation of", n2, "is", oct(n2))
print("Hexadecimal representation of", n2, "is", hex(n2))
print("Complex representation of", n1, "is", complex(n2))
```

```
python example5.py 10
# Original number is 10
# Binary representation of 10 is 0b1010
# Octal representation of 10 is 0o12
# Hexadecimal representation of 10 is 0xa
# Complex representation of 10 is (10+0j)
```

17 Create the parameterized job

New Item → `example5` → Freestyle → OK → tick **This project is parameterized** → **String Parameter** Name: `num` → Build Steps → Execute Windows batch command:

```
cd /d C:\jenkinsprojects
python example5.py %num%
```

→ Save → **Build with Parameters** → num = `10` → Build → Console Output shows the same 5 lines and `Finished: SUCCESS`.

If asked about GitHub Actions (title says Jenkins/GitHub Actions)

18 Point to Experiment 1

Your CI with GitHub Actions is the `.github/workflows/student-ci.yml` pipeline from Experiment 1 (checkout → set up Python → install dependencies → run tests on every push/PR). Explain the difference: Jenkins = self-hosted server you configure; GitHub Actions = CI built into GitHub, defined in YAML.

Show the examiner:

- Each job's configuration (build step / parameters) and its Console Output ending in Finished: SUCCESS.
- Build with Parameters page with your values (String + Choice).
- The program tested locally in Command Prompt first, then in Jenkins.

Common errors & fixes

Problem	Fix
“python/javac is not recognized” in Jenkins but works in CMD	Jenkins runs as SYSTEM: add Python/JDK to the System PATH and restart the Jenkins service (or use full paths, e.g. C:\Users\<you>\AppData\Local\Programs\Python\Python312\python.exe).
“The system cannot find the path specified”	Folder name wrong — check C:\jenkinsprojects exists; use cd /d to change drive.
“can't open file … No such file or directory”	File name typo (e.g. example4.py vs example5.py) — run dir to list files.
Build with Parameters option missing	Tick “This project is parameterized” in General and Save.
Java: “ArrayIndexOutOfBoundsException”	No argument passed — num parameter empty or not used (%num%).
“Error: Could not find or load main class”	Class name must match: java mul (not mul.java); compile first with javac.

Viva questions

Q. What is Continuous Integration?
Developers integrate code frequently into a shared repository; each integration is automatically built and tested to catch errors early.

Q. What is Jenkins?
An open-source automation server for building, testing and deploying software — runs on port 8080, extended with plugins.

Q. What is a Freestyle project?
The simplest Jenkins job type, configured through the UI to run build steps such as batch/shell commands and programs.

Q. What is a parameterized build?
A build that takes input values at runtime (String, Choice, Boolean parameters) which are used in the build commands — %name% on Windows, $name on Linux.

Q. Execute shell vs Execute Windows batch command?
Shell runs Linux/macOS sh commands; batch runs Windows cmd commands.

Q. Where does Jenkins store job workspaces on Windows?
C:\ProgramData\Jenkins\.jenkins\workspace\<job-name> (shown in Console Output).

Q. Freestyle vs Pipeline job?
Freestyle is configured in the UI; Pipeline is defined as code in a Jenkinsfile with stages.

Conclusion: This experiment demonstrated how Jenkins can automate the execution of programs on Windows. Freestyle projects were created to run batch commands, Java and Python programs, and parameterized builds passed runtime values to the programs. Thus, Jenkins automates the build and execution process efficiently, which is the basis of Continuous Integration.

Experiment 4

To containerize applications using Docker

Aim: To containerize applications using Docker — understand Docker architecture and the container lifecycle, and use essential Docker commands to manage images and interact with containers.

Workflow: docker pull (image) → docker run (create + start container) → docker ps → docker stop → docker rm → docker image rm

Requirements

- Windows 10/11 64-bit with virtualization enabled in BIOS.
- Docker Desktop + WSL 2.
- Command Prompt or PowerShell (two windows open).

Procedure

Part A — Install Docker Desktop (as in your journal)

1 Download and install

Open **docker.com** → **Get started** → **Docker Desktop** → **Download for Windows** → run **Docker Desktop Installer** → keep *Use WSL 2* ticked → install → **restart** the PC.

2 Finish WSL 2 setup

Open Docker Desktop → **Accept** the terms. If you see *"WSL 2 installation is incomplete"*: open the link **https://aka.ms/wsl2kernel** → download and run the **WSL2 Linux kernel update package for x64** (.msi) → Next → Finish. Then open **PowerShell as Administrator** and run:

```
wsl --set-default-version 2
```

Click **Restart** in the Docker window → Docker restarts → click **Start** → the Docker Desktop dashboard (*No containers running*) means installation is complete.

Tip: On newer Windows you can simply run `wsl --install` or `wsl --update` in admin PowerShell instead of the .msi.

Part B — Docker commands (main practical)

3 Check the version

Make sure Docker Desktop is running (whale icon, *Engine running*). Open Command Prompt:

```
docker --version              # Docker version 29.x.x, build ...
```

4 Pull the Ubuntu image from Docker Hub

```
docker pull ubuntu            # Using default tag: latest ... Status: Downloaded newer image
for ubuntu:latest
```

5 List downloaded images

```
docker images                 # REPOSITORY/IMAGE  TAG  IMAGE ID  SIZE   -> ubuntu  latest
3131b4cc82a7 ...
```

6 Run Ubuntu as a container (interactive)

```
docker run -it ubuntu /bin/bash
# prompt changes to:  root@dbfaed403752:/#    <- you are now INSIDE the Ubuntu container
```

`-i` = interactive, `-t` = terminal, `/bin/bash` = start a bash shell. Try a few Linux commands inside (good to show the examiner):

```
ls
cat /etc/os-release           # shows Ubuntu version
whoami                        # root
exit                          # leaves and stops the container
```

> **Tip:** For the next step keep this container running — do not type exit yet; open a **second** Command Prompt.

7 Manage containers from a second Command Prompt

```
docker ps                          # running containers only
docker container ls -a             # ALL containers (running + exited)
docker container stop <container-id>
docker container rm <container-id>
docker ps                          # verify it is gone
```

Copy the **CONTAINER ID** (e.g. `60fd46de8c04`) from the previous output — use **your own** ID. Typing just the first 3–4 characters is enough.

8 Remove an image

```
docker                             # lists all docker commands (help)
docker images
docker image rm <image-id>         # e.g. docker image rm 96498ffd522e  (hello-world)
docker images                      # verify
```

> **Watch out:** An image cannot be removed while any container (even a stopped one) uses it — remove those containers first (`docker container ls -a`, then `docker container rm <id>`), or use `docker image rm -f`.

9 Run Ubuntu again and check

```
docker run -it ubuntu /bin/bash    # new container from the same image (new ID)
```

This shows the container **lifecycle**: create → start/run → stop → remove, and that one image can create many containers.

Part C — Containerize your own app (only if the examiner asks)

10 Create app files and a Dockerfile

Folder `docker-demo` with:

app.py

```
from flask import Flask

app = Flask(__name__)


@app.route("/")
def home():
    return "Hello from a Docker container!"


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

requirements.txt

```
flask
```

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

11 Build, run, verify

```
docker build -t flask-app:1.0 .
docker images
docker run -d -p 5000:5000 --name myflask flask-app:1.0
docker ps
```

Open **http://localhost:5000** → *Hello from a Docker container!*. Stop/remove: `docker rm -f myflask`.

Show the examiner:

- docker --version, docker images (ubuntu listed).
- The root@<id>:/# prompt inside the Ubuntu container.
- docker ps / docker container ls -a, then stop, rm and image rm with your own IDs.
- Docker Desktop dashboard showing the container.

Common errors & fixes

Problem	Fix
“error during connect … docker daemon is not running”	Start Docker Desktop and wait for “Engine running”.
“WSL 2 installation is incomplete”	Install the WSL2 kernel update (aka.ms/wsl2kernel) or run wsl --update, then restart Docker.
“Virtualization not enabled”	Enable Intel VT-x / AMD-V (SVM) in BIOS.
“image is being used by stopped container”	Remove that container first, or docker image rm -f <id>.
“No such container”	Wrong/old ID — copy the ID again from docker container ls -a.

docker pull very slow / fails	Check internet; lab proxy may block Docker Hub — try mobile hotspot.

Viva questions

Q. Difference between containerization and virtualization? (journal Q1)
Containers share the host OS kernel — lightweight, start in seconds; virtual machines run a full guest OS on a hypervisor — heavier and slower (e.g. VMware, VirtualBox).

Q. What is the Docker Daemon? (journal Q2)
dockerd — the background service that manages images, containers, networks and volumes; it receives requests from the Docker client through the Docker API.

Q. Image vs container?
Image = read-only template; container = running instance of an image.

Q. Name the Docker architecture components.
Docker Client, Docker Daemon (host), Docker Registry (Docker Hub), Images, Containers.

Q. What do -i and -t mean in docker run -it?
-i keeps input open (interactive), -t allocates a terminal.

Q. docker ps vs docker ps -a?
ps shows only running containers; -a shows all, including exited ones.

Q. What is a Dockerfile?
A text file of instructions (FROM, COPY, RUN, CMD…) used by docker build to create an image.

> **Conclusion:** Docker Desktop was installed with WSL 2, and essential Docker commands were used to pull images, run containers interactively, list, stop and remove containers, and remove images. This demonstrated Docker architecture and the container lifecycle, enabling applications to be packaged and run consistently across environments.

Experiment 7

To deploy a serverless function using AWS Lambda

Aim: To deploy a serverless function using AWS Lambda by creating, configuring, testing and executing a Lambda function in the AWS Cloud environment, and monitoring it using Amazon CloudWatch.

Workflow: Create Function → Select Runtime → Write Code → Deploy → Test (event) → Monitor (CloudWatch)

Requirements

- AWS account (Free Tier) — log in to the **AWS Management Console** before the exam.
- Any region works — your journal used **Europe (Stockholm) eu-north-1**.

Procedure

Part A — Create the function

1 Open AWS Lambda

Log in to the AWS Console → check the **region** (top-right, e.g. Europe (Stockholm) or Asia Pacific (Mumbai)) → search **Lambda** → open it → **Create a function**.

2 Basic information

Select **Author from scratch** →
• Function name: `RashmitLambdaFunction`
• Runtime: **Python 3.14** (latest Python offered)
• Architecture: x86_64
• Permissions: leave default — Lambda **creates an execution role with permission to upload logs to CloudWatch**
→ **Create function**.

Expected output: green banner *Successfully created the function 'RashmitLambdaFunction'*; the Function overview and Code source editor appear.

Watch out: If your college uses **AWS Academy Learner Lab**: Permissions → **Use an existing role → LabRole** (you are not allowed to create roles there).

3 Write the code

In **Code source**, open `lambda_function.py` and replace everything with:

lambda_function.py

```
def lambda_handler(event, context):
    name = event.get("name", "Student")

    return {
        "statusCode": 200,
        "body": f"Hello {name}! AWS Lambda function executed successfully."
    }
```

`event` = input data that triggered the function; `context` = runtime information.
`event.get("name", "Student")` uses "Student" if no name is sent.

4 Deploy

Click **Deploy** (left panel, or **Ctrl+Shift+U**).

> **Expected output:** *Successfully updated the function 'RashmitLambdaFunction'.*

> **Watch out:** Changes are not live until you click Deploy.

Part B — Test with a sample event

5 Create a test event

Left panel **TEST EVENTS → Create new test event** (or the **Test** tab) → Event name `RashmitTestEvent` → Invocation type **Synchronous** → Event JSON:

```
{
  "name": "Rashmit"
}
```

→ **Save**.

6 Run the test

Click **Test** (or **Ctrl+Shift+I**). The OUTPUT panel shows the Execution Results:

```
Status: Succeeded
Test Event Name: RashmitTestEvent

Response:
{
  "statusCode": 200,
  "body": "Hello Rashmit! AWS Lambda function executed successfully."
}
```

Below it: **Function Logs** with `START RequestId`, `END`, `REPORT … Duration … Billed Duration … Memory Size: 128 MB … Max Memory Used`.

7 Show event-driven behaviour (optional, impressive)

Create a second test event with `{}` (no name) → Test → body becomes `Hello Student! …`. Same function, different event → different result.

Part C — Monitor using Amazon CloudWatch

8 Open the logs

Function page → **Monitor** tab → **View CloudWatch logs** → log group `/aws/lambda/RashmitLambdaFunction` → open the latest **log stream**.

> **Expected output:** Log events: `INIT_START Runtime Version: python:3.14 …` (cold start), `START RequestId …`, `END RequestId …`, `REPORT … Duration: 1.60 ms Billed Duration: … Memory Size: 128 MB`.

9 Metrics

Back on the **Monitor** tab: graphs for **Invocations**, **Duration**, **Error count and success rate**, **Throttles** (click refresh after a few test runs).

Part D — Optional extras and clean-up

10 Invoke over HTTP (if asked how to trigger from a browser)

Configuration → Function URL → Create function URL → Auth type **NONE** → Save → open the URL in a browser → `Hello Student! …`. (Via URL the name comes in the query string, so this

simple code always shows the default “Student”.) Other triggers: **+ Add trigger** → API Gateway, S3, EventBridge.

11 Clean up

After the exam: **Actions → Delete** the function so nothing keeps running.

Show the examiner:

- Function created with runtime Python 3.14 and the code deployed.
- Test event RashmitTestEvent → Status: Succeeded with statusCode 200 and the response body.
- CloudWatch log group /aws/lambda/RashmitLambdaFunction with START / END / REPORT lines.

Common errors & fixes

Problem	Fix
Edited code but output unchanged	Click Deploy before Test.
“Runtime.HandlerNotFound”	File must be lambda_function.py with function lambda_handler (handler = lambda_function.lambda_handler).
“Task timed out after 3.00 seconds”	Configuration → General configuration → Edit → increase Timeout.
Test JSON error	Event must be valid JSON — double quotes, no trailing comma.
No logs in CloudWatch	Wait a few seconds and refresh; the execution role needs CloudWatch Logs permission (default role has it).
Can't find the function	You are in a different region — switch region top-right.

Viva questions

Q. What is AWS Lambda? (journal Q1)
A serverless computing service that runs your code in response to events without managing servers or infrastructure.

Q. Purpose of Amazon CloudWatch in Lambda? (journal Q2)
To monitor Lambda functions and view execution logs, duration, memory usage and errors.

Q. What is serverless computing?
Running code without managing servers — the provider handles provisioning, scaling and availability; you pay per use.

Q. What are event and context?
event = input data from the trigger; context = runtime info like request ID and remaining time.

Q. What is a cold start?
Extra startup delay when Lambda creates a new execution environment (INIT_START / Init Duration in the logs).

Q. Name event sources for Lambda.
API Gateway, S3, SQS, DynamoDB, EventBridge (schedules), Function URL.

Q. Max timeout and memory?
Timeout up to 15 minutes; memory 128 MB to 10 GB.

Q. Synchronous vs asynchronous invocation?

Synchronous waits for the response (console test, API); asynchronous queues the event and returns immediately with a request ID.

> **Conclusion:** The serverless function was successfully created, configured, deployed and tested using AWS Lambda. The function executed successfully using a test event, and its execution was verified through Amazon CloudWatch logs. This experiment demonstrated how AWS Lambda enables serverless, event-driven application execution without managing servers.